

Resenhômetro — Guia da Equipe

É os mais bigodes da capital krai.

Documento para onboarding da equipe — README do projeto + instalação + Git básico.

Sobre o projeto

O **Resenhômetro** é uma plataforma para facilitar a organização de encontros entre amigos.

O sistema permitirá criar rolês, confirmar presença, compartilhar localização, comentar, avaliar encontros e centralizar todas as informações importantes em um único lugar.

Funcionalidades

- Cadastro e Login
- Perfil de usuário
- Criar, editar e excluir rolês
- Localização do encontro
- Comentários e avaliações
- Confirmar presença / informar ausência
- Informar gastos
- Fotos e notificações (futuro)

Arquitetura

```
Usuário → Web (Next.js) → API (Fastify) → PostgreSQL
                        ↓
                    OpenStreetMap / JWT
```

Fluxo do backend: Request → Controller → Service → Repository → Database → Response

Stack de tecnologias

Front-end: Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, React Hook Form, Zod, Axios

Back-end: Node.js, Fastify, TypeScript, Drizzle ORM, JWT, bcrypt

Banco: PostgreSQL

Ferramentas: Docker, GitHub, Bruno, VS Code / Cursor

Estrutura do projeto

```
resenhometro/
├── apps/
│   ├── api/      → Backend (Fastify)
│   ├── web/      → Frontend (Next.js)
│   └── mobile/   → Futuro
```

```
|   └─ desktop/    → Futuro
├─ packages/
|   └─ ui/         → Componentes reutilizáveis
|   └─ shared/     → Código compartilhado
|   └─ config/     → Configurações
├─ docs/
├─ docker/
└─ .github/
```

Equipe

Jean, João, Adryan, Rafael, Lucas, Niel, Davi, Gabriel

PARTE 1 — Instalação das ferramentas

Instale **uma única vez** na sua máquina:

Ferramenta	Para quê?	Link
Git	Clonar e versionar código	git-scm.com/downloads
Node.js 20+	Rodar JavaScript/TypeScript	nodejs.org
pnpm 10+	Dependências do monorepo	pnpm.io/installation
Docker Desktop	PostgreSQL local	docker.com/products/docker-desktop

Verificar instalação

```
git --version
node --version
pnpm --version
docker --version
```

Instalar pnpm

```
npm install -g pnpm
```

PARTE 2 — Clonar e rodar o projeto

1. Clonar o repositório

```
git clone https://github.com/Jean-Griggi/radar-de-resenha.git
cd radar-de-resenha
```

2. Instalar dependências

```
pnpm install
```

Baixa automaticamente Next.js, Fastify, React, TypeScript, etc.

3. Configurar ambiente

Windows (PowerShell):

```
Copy-Item .env.example .env
```

Linux / macOS:

```
cp .env.example .env
```

4. Subir o banco (Docker aberto)

```
docker compose -f docker/docker-compose.yml up -d
```

5. Rodar o projeto

```
pnpm dev
```

App	URL
Web	http://localhost:3000
API	http://localhost:3333

Testar API: `curl http://localhost:3333/health`

Comandos do projeto

Comando	O que faz
<code>pnpm install</code>	Instala dependências
<code>pnpm dev</code>	Sobe web + api
<code>pnpm build</code>	Build de produção
<code>pnpm lint</code>	Verifica código
<code>pnpm typecheck</code>	Verifica TypeScript

pnpm format

Formata com Prettier

Problemas comuns

Problema	Solução
pnpm não encontrado	<code>npm install -g pnpm</code>
node não encontrado	Instale Node.js 20+
Erro no banco	Verifique se o Docker está rodando
Porta em uso	Feche o processo ou mude no <code>.env</code>

PARTE 3 — Git e GitHub (tutorial básico)

O que é Git?

Git é um sistema de **controle de versão**. Ele guarda o histórico do código e permite várias pessoas trabalharem juntas sem sobrescrever o trabalho uma da outra.

O que é GitHub?

GitHub é onde o repositório fica **** hospedado na nuvem****. Vocês clonam de lá, fazem push das alterações e abrem Pull Requests.

Configuração inicial (fazer uma vez)

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@gmail.com"
```

Comandos Git essenciais

Comando	O que faz
<code>git status</code>	Mostra arquivos alterados
<code>git add .</code>	Prepara todos os arquivos para commit
<code>git add arquivo.ts</code>	Prepara um arquivo específico
<code>git commit -m "mensagem"</code>	Salva um snapshot das alterações
<code>git push</code>	Envia commits para o GitHub
<code>git pull</code>	Baixa atualizações do GitHub
<code>git log --oneline</code>	Mostra histórico de commits

git diff	Mostra o que mudou
----------	--------------------

Fluxo do dia a dia

```
# 1. Atualizar sua branch
git pull

# 2. Trabalhar no código...

# 3. Ver o que mudou
git status

# 4. Adicionar e commitar
git add .
git commit -m "feat: descreva o que fez"

# 5. Enviar para o GitHub
git push
```

Branches (ramificações)

Branch = linha de desenvolvimento separada. Evita que todo mundo mexa direto na `main`.

Estrutura do projeto

```
main
├── develop
│   ├── feature/login
│   ├── feature/roles
│   └── fix/bug-x
```

Criar e usar uma branch

```
# Ver branch atual
git branch

# Criar branch nova
git checkout -b feature/nome-da-feature

# Exemplo
git checkout -b feature/login

# Enviar branch para o GitHub (primeira vez)
git push -u origin feature/login
```

Voltar para outra branch

```
git checkout develop
git checkout main
```

Atualizar sua branch com a develop

```
git checkout feature/sua-feature
git pull origin develop
```

Padrão de nomes de branch

feature/nome-da-feature	→ nova funcionalidade
fix/nome-do-bug	→ correção de bug
hotfix/nome	→ correção urgente em produção
docs/documentacao	→ documentação
refactor/refatoracao	→ refatoração de código

Exemplos:

- feature/login
- feature/criar-roles
- fix/autenticacao-jwt

Padrão de commits (Conventional Commits)

Prefixo	Quando usar
feat:	Nova funcionalidade
fix:	Correção de bug
docs:	Documentação
style:	Formatação (sem mudar lógica)
refactor:	Refatoração
test:	Testes
chore:	Manutenção, configs

Exemplos:

```
feat: cria sistema de login
fix: corrige autenticação JWT
docs: atualiza README
refactor: reorganiza estrutura da API
```

Pull Request (PR)

Depois de terminar uma feature:

1. Faça push da sua branch
2. No GitHub, clique em **"Compare & pull request"**
3. Descreva o que foi feito
4. Peça review de um colega
5. Após aprovação, faça merge na `develop`

Fluxo recomendado para a equipe

- ```
1. git checkout develop
2. git pull
3. git checkout -b feature/minha-task
4. (codar...)
5. git add .
6. git commit -m "feat: minha alteração"
7. git push -u origin feature/minha-task
8. Abrir Pull Request no GitHub → develop
```

**Importante:** evitem commitar direto na `main`. Usem branches e PRs.

---

## Objetivos do projeto

- Aprender novas tecnologias
- Praticar desenvolvimento em equipe
- Criar um projeto real
- Melhorar conhecimentos em arquitetura
- Evoluir em Git e GitHub
- Desenvolver um sistema útil para o grupo

---

**Licença:** Projeto privado — uso interno da equipe.

Gerado para a equipe Resenhômetro — 2026